

# ■ Complete Exam Study Guide

Object Oriented Programming Using Java - II

T.Y.B.Sc Computer Science | Internal Test (15 Marks)

---

## Q.1 — One Mark Questions (Solve Any 5)

### a) Interfaces Implemented by TreeSet

TreeSet implements: **Set, SortedSet, and NavigableSet** interfaces (also indirectly implements Iterable and Collection).

**Definition:** TreeSet stores elements in **sorted ascending order** using a balanced binary tree. It does **not allow duplicate elements**.

**One-line answer:** TreeSet implements Set, SortedSet, and NavigableSet interfaces.

### b) ArrayList is a List Implemented Using a Resizable Array — Comment

- ArrayList is a class in **java.util** package that implements the **List interface**.
- Internally, it uses a **dynamic/resizable array** to store elements.
- When the array becomes full, Java **automatically increases the size** by creating a larger array and copying elements.
- It allows **duplicate elements** and maintains **insertion order**.
- Accessing elements is fast (by index), but **adding/removing in the middle is slow** because elements need to shift.
- **Conclusion:** Yes, ArrayList IS a resizable array — you don't declare size in advance; it grows and shrinks automatically.

### c) Use of Collection Interface

**Definition:** The Collection interface is the **root interface** of the Java Collections Framework, in the **java.util** package.

**Uses / Purpose:**

- Provides basic operations: add, remove, search, and iterate on a group of objects.
- Defines common methods used by all collection classes (ArrayList, HashSet, TreeSet, etc.).
- Allows manipulation of a group of objects as a single unit.
- Helps achieve code reusability.

**Key Methods:** add(), remove(), contains(), size(), iterator(), clear()

### d) Two Ways of Creating a Thread

#### Way 1: Extending the Thread class

```
class MyThread extends Thread {  
    public void run() { System.out.println("Thread running"); }  
}  
// In main:
```

```
MyThread t = new MyThread(); t.start();
```

### Way 2: Implementing the Runnable interface

```
class MyRunnable implements Runnable {  
    public void run() { System.out.println("Thread running"); }  
}  
// In main:  
Thread t = new Thread(new MyRunnable()); t.start();
```

**Key Difference:** Way 1 cannot extend another class. Way 2 (Runnable) is more flexible as the class can still extend another class.

## e) Two Methods of Inter-Thread Communication

**Definition:** Inter-thread communication allows threads to cooperate/communicate while sharing resources.

| Method      | Description                                                              |
|-------------|--------------------------------------------------------------------------|
| wait()      | Makes current thread pause and release the lock until notify() is called |
| notify()    | Wakes up ONE waiting thread that called wait() on the same object        |
| notifyAll() | Wakes up ALL waiting threads that called wait() on the same object       |

**Exam answer (two methods):** wait() and notify()

## f) Difference Between Iterator and ListIterator

| Feature               | Iterator                    | ListIterator                                               |
|-----------------------|-----------------------------|------------------------------------------------------------|
| Direction             | Forward only                | Forward AND Backward                                       |
| Applicable to         | Any Collection              | Only List (ArrayList, LinkedList)                          |
| Can add elements?     | No                          | Yes — add() method                                         |
| Can replace elements? | No                          | Yes — set() method                                         |
| Methods               | hasNext(), next(), remove() | hasNext(), next(), hasPrevious(), previous(), add(), set() |

## Q.2 — 5 Mark Programs (Solve Any 2)

### a) Store 'n' Employee Info in Hashtable and Display

```
import java.util.*;  
  
class EmployeeHashtable {  
    public static void main(String args[]) {  
        Scanner sc = new Scanner(System.in);  
        Hashtable<Integer, String> ht = new Hashtable<Integer, String>();  
  
        System.out.print("Enter number of employees: ");  
        int n = sc.nextInt();  
  
        for (int i = 1; i <= n; i++) {  
            System.out.print("Enter Employee ID: ");  
            int id = sc.nextInt();  
            System.out.print("Enter Employee Name: ");
```

```

        String name = sc.next();
        ht.put(id, name);
    }

    System.out.println("--- Employee Details ---");
    System.out.println("ID\tName");
    Set<Integer> keys = ht.keySet();
    for (Integer key : keys) {
        System.out.println(key + "\t" + ht.get(key));
    }
    sc.close();
}
}

```

**Output:** Enter number of employees: 3 | Enter Employee ID: 101 | Enter Employee Name: Rahul ...

--- Employee Details ---

ID Name

103 Amit 102 Priya 101 Rahul

## b) Display 'examination' 50 Times Using Multithreading

```

class PrintThread extends Thread {
    public void run() {
        for (int i = 1; i <= 50; i++) {
            System.out.println(i + ": examination");
        }
    }
}

class MultiThreadDemo {
    public static void main(String args[]) {
        PrintThread t = new PrintThread();
        t.start();
    }
}

```

**Output:** 1: examination 2: examination ... 50: examination

## c) LinkedList of 4 Integers — Add at First + Delete Last

```

import java.util.*;

class LinkedListDemo {
    public static void main(String args[]) {
        LinkedList<Integer> ll = new LinkedList<Integer>();
        ll.add(10);
        ll.add(20);
        ll.add(30);
        ll.add(40);

        System.out.println("Original List: " + ll);

        // I] Add element at first position
        ll.addFirst(5);
        System.out.println("After adding 5 at first: " + ll);

        // II] Delete last element
    }
}

```

```

    ll.removeLast();
    System.out.println("After deleting last element: " + ll);
}
}

```

#### Output:

Original List: [10, 20, 30, 40]

After adding 5 at first: [5, 10, 20, 30, 40]

After deleting last element: [5, 10, 20, 30]

## Quick Revision Table

| Topic                           | Key Point                                                      |
|---------------------------------|----------------------------------------------------------------|
| <b>TreeSet interfaces</b>       | Set, SortedSet, NavigableSet                                   |
| <b>ArrayList</b>                | Resizable array, allows duplicates, maintains insertion order  |
| <b>Collection Interface</b>     | Root interface; provides add, remove, iterate, size methods    |
| <b>Thread creation</b>          | 1. Extend Thread class 2. Implement Runnable interface         |
| <b>Inter-thread methods</b>     | wait(), notify(), notifyAll() — defined in Object class        |
| <b>Iterator vs ListIterator</b> | Iterator = forward only; ListIterator = both directions        |
| <b>Hashtable</b>                | Stores key-value pairs; use put() to insert, get() to retrieve |
| <b>LinkedList methods</b>       | addFirst(), removeLast(), addLast(), removeFirst()             |

## Key Definitions (One-Liners)

**Thread:** A lightweight sub-process; the smallest unit of processing.

**Multithreading:** Running two or more threads simultaneously in a single program.

**Collection:** A group of objects stored and manipulated as a single unit.

**TreeSet:** A Set that stores elements in sorted ascending order using a tree.

**ArrayList:** A List backed by a dynamic resizable array.

**Hashtable:** Stores data as key-value pairs; thread-safe; no null keys.

**LinkedList:** A doubly-linked list allowing add/remove at any position.

**Iterator:** Used to loop through a collection in the forward direction only.

**ListIterator:** Like Iterator, but works in both directions on a List.